P3179R2
C++ parallel range algorithms

Published Proposal, 2024-06-25

This version:
https://wg21.link/P3179R2

Authors:
Ruslan Arutyunyan (Intel)
Alexey Kukanov (Intel)
Bryce Adelstein Lelbach (he/him/his) (Nvidia)

Audience:
SG9, SG1

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper proposes adding parallel algorithms that work together with the C++ Ranges library.

Table of Contents

§ 1. Motivation

Standard parallel algorithms with execution policies which set semantic requirements to user-provided callable objects were a good start for supporting parallelism in the C++ standard.

The C++ Ranges library - ranges, views, etc. - is a powerful facility to produce lazily evaluated pipelines that can be processed by range-based algorithms. Together they provide a productive and expressive API with the room for extra optimizations.

Combining these two powerful features by adding support for execution policies to the range-based algorithms opens an opportunity to fuse several computations into one parallel algorithm call, thus reducing the overhead on parallelism. That is especially valuable for heterogeneous implementations of parallel algorithms, for which the range-based API helps reducing the number of kernels submitted to an accelerator.

Users are already using ranges and range adaptors by passing range iterators to the existing non-range parallel algorithms. [P2408R5] was adopted to enable this. This pattern is often featured when teaching C++ parallel algorithms and appears in many codebases.

`iota` and `cartesian_product` are especially common, as many compute workloads want to iterate over indices, not objects, and many work with multidimensional data. `transform` is also common, as it enables fusion of element-wise operations into a single parallel algorithm call, which can avoid the need for temporary storage and is more performant than two separate calls.

However, passing range iterators to non-range algorithms is unwieldy and verbose. It is surprising to users that they cannot simply pass the ranges to the parallel algorithms as they would for serial algorithms.

Scalar-Vector Multiply

Before	After

`std::span data = …;`
`double C = …;`
 	
`auto indices = std::views::iota(1, data.size());`
`std::for_each(std::execution::par_unseq,`
`  std::ranges::begin(indices),`
`  std::ranges::end(indices),`
`  [=] (auto i) { data[i] *= C; });`
	`std::span data = …;`
	`double C = …;`
	
	`std::for_each(std::execution::par_unseq,`
	`  std::views::iota(1, data.size()),`
	`  [=] (auto i) { data[i] *= C; });`

Matrix Transpose	
Before	**After**

`std::mdspan A{input,  N, M};`
`std::mdspan B{output, M, N};`
 	
`auto indices = std::views::cartesian_product(`
`  std::views::iota(0, A.extent(0)),`
`  std::views::iota(0, A.extent(1)));`
 	
`std::for_each(std::execution::par_unseq,`
`  std::ranges::begin(indices),`
`  std::ranges::end(indices),`
`  [=] (auto idx) {`
`  auto [i, j] = idx;`
`  B[j, i] = A[i, j];`
`  });`
	`std::mdspan A{input,  N, M};`
	`std::mdspan B{output, M, N};`
	
	`std::for_each(std::execution::par_unseq,`
	`  std::views::cartesian_product(`
	`  std::views::iota(0, A.extent(0)),`
	`  std::views::iota(0, A.extent(1))),`
	`  [=] (auto idx) {`
	`  auto [i, j] = idx;`
	`  B[j, i] = A[i, j];`
	`  });`

Earlier, [P2500R2] proposed to add the range-based C++ parallel algorithms together with its primary goal of extending algorithms with schedulers. We have decided to split those parts to separate papers, which could progress independently.

§ 2. Design overview

This paper proposes execution policy support for C++ range-based algorithms. In the nutshell, the proposal extends C++ range algorithms with overloads taking any standard C++ execution policy as a function parameter. These overloads are further referred to as *parallel range algorithms*.

The proposal is targeted to C++26.

§ 2.1. Design summary

§ 2.1.1. Differences to serial range algorithms

Comparing to the C++20 serial range algorithms, we propose the following modifications:

- The execution policy parameter is added.
- `for_each` and `for_each_n` return only an iterator but not the function.

- Parallel range algorithms take `range`, not an iterator, as an output for the overloads with ranges, and additionally take an output sentinel for the "iterator and sentinel" overloads. (§ 2.6 Taking range as an output)
- Until better parallelism-friendly abstractions are proposed, parallel algorithms require `random_access_{iterator,range}`. (§ 2.5 Requiring random_access_iterator or random_access_range)
- At least one of the input sequences as well as the output sequence must be bounded. (§ 2.7 Requiring ranges to be bounded)

§ 2.1.2. Differences to C++17 parallel algorithms

In addition to data sequences being passed as either ranges or "iterator and sentinel" pairs, the following differences to the C++17 parallel algorithms are proposed:

- `for_each` returns an iterator, not `void`.
- Algorithms require `random_access_{iterator,range}`, and not *LegacyForwardIterator*.
- At least one of the input sequences as well as the output sequence must be bounded.

§ 2.1.3. Other design aspects

- Except as mentioned above, the parallel range algorithms should return the same type as the corresponding serial range algorithms. (§ 2.3 Algorithm return types)
- The proposed algorithms should follow the design of serial range algorithms with regard to name lookup. (§ 2.4 Non ADL-discoverable functions)
- The proposed algorithms should require callable object passed to an algorithm to be `regular_invocable` where possible. (§ 2.8 Requirements for callable parameters)
- The proposed APIs are not customization points. (§ 2.9 Parallel range algorithms are not customization points)
- The proposed algorithms should follow the design of C++17 parallel algorithms with regard to `constexpr` support. (§ 2.10 constexpr parallel range algorithms)

§ 2.2. Coexistence with schedulers

We believe that adding parallel range algorithms does not have the risk of conflict with anticipated scheduler-based algorithms, because an execution policy does not satisfy the requirements for a policy-aware scheduler ([P2500R2]), a sender ([P3300R0]), or really anything else from [P2300R9] that can be used to specify such algorithms.

At this point we do not, however, discuss how the appearance of schedulers may or should impact the execution rules for parallel algorithms specified in [algorithms.parallel.exec], and just assume that the same rules apply to the range algorithms with execution policies.

§ 2.3. Algorithm return types

We explored possible algorithm return types and came to conclusion that returning the same type as serial range algorithms is the preferred option to make the changes for enabling parallelism minimal.

```
auto res = std::ranges::sort(v);
```

becomes:

```
auto res = std::ranges::sort(std::execution::par, v);
```

However, `std::ranges::for_each` and `std::ranges::for_each_n` require special consideration because previous design decisions suggest that there should be a difference between serial and parallel versions.

The following table summarizes return value types for the existing variants of these two algorithms:

API	Return type
`std::for_each`	`Function`
Parallel `std::for_each`	`void`
`std::for_each_n`	`Iterator`
Parallel `std::for_each_n`	`Iterator`
`std::ranges::for_each`	`for_each_result<ranges::borrowed_iterator_t<Range>, Function>`
`std::ranges::for_each`, `I` + `S` overload	`for_each_result<Iterator, Function>`
`std::ranges::for_each_n`	`for_each_n_result<Iterator, Function>`

While the serial `std::for_each` returns the obtained function object with all modifications it might have accumulated, the return type for the parallel `std::for_each` is `void` because, as stated in the standard, "parallelization often does not permit efficient state accumulation". For efficient parallelism an implementation can make multiple copies of the function object, which for that purpose is allowed to be copyable and not just movable like for the serial `for_each`. That implies that users cannot rely on any state accumulation within that function object, so it does not make sense (and might be even dangerous) to return it.

In `std::ranges`, the return type of `for_each` and `for_each_n` is unified to return both an iterator and the function object.

Based on the analysis above and the feedback from SG9 we think that the most reasonable return type for parallel variants of `std::ranges::for_each` and `std::ranges::for_each_n` should be:

API	Return type
Parallel `std::ranges::for_each`	`ranges::borrowed_iterator_t<Range>`
Parallel `std::ranges::for_each`, `I` + `S` overload	`Iterator`
Parallel `std::ranges::for_each_n`	`Iterator`

§ 2.4. Non ADL-discoverable functions

We believe the proposed functionality should have the same behavior as serial range algorithms regarding the name lookup. For now, the new overloads are supposed to be special functions that are not discoverable by ADL (the status quo of the standard for serial range algorithms).

[P3136R0] suggests to respecify range algorithms to be actual function objects. If adopted, that proposal will apply to all algorithms in the `std::ranges` namespace, thus automatically covering the parallel algorithms we propose.

Either way, adding parallel versions of the range algorithms should not be a problem. Please see § 4.1 Possible implementation of a parallel range algorithm for more information.

§ 2.5. Requiring `random_access_iterator` or `random_access_range`

C++17 parallel algorithms minimally require *LegacyForwardIterator* for data sequences, but in our opinion, it is not quite suitable for an efficient parallel implementation. Therefore for parallel range algorithms we propose to require random access ranges and iterators.

Though the feedback we received in Tokyo requested to support forward ranges, we would like this question to be discussed in more detail. Using parallel algorithms with forward ranges will in most cases give little to no benefit, and may even reduce performance due to extra overheads. We believe that forward ranges and iterators are bad abstractions for parallel data processing, and allowing those could result in wrong expectations and unsatisfactory user experience with parallel algorithms.

Many parallel programming models that are well known and widely used in the industry, including OpenMP, OpenCL, CUDA, SYCL, oneTBB, define iteration or data spaces for their parallel constructs in ways that allow creating sufficient parallel work quickly and efficiently. A key property for this is the ability to split the work into smaller chunks. These programming models allow to control the amount of work per chunk and sometimes the ways chunks are created and/or scheduled. All these also support iteration spaces up to at least 3 dimensions.

Except for `tbb::parallel_for_each` in oneTBB which can work with forward iterators, these parallel programming models require random access iterators or some equivalent, such as numeric indexes or pointers. This is natural, as referring to an arbitrary point in the iteration space at constant time is the main and by far simplest way to create parallel work. Forward iterators, on the other hand, are notoriously bad for splitting a sequence that can only be done in linear time. Moreover, if the output of an algorithm should preserve the order of its input, which is typical for the C++ algorithms, it requires additional synchronization or/and additional space with forward iterators and comes almost for granted with random access ones.

These very programming models are often used as backends to implement the C++ standard parallelism. Not surprisingly, most implementations fall back to serial processing if data sequences have no random access. Of the GNU libstdc++, LLVM libc++, and MSVC's standard library, only the latter attempts to process forward iterator based sequences in parallel, for which it first needs to serially iterate over a whole sequence once or even twice. oneAPI Data Parallel C++ library (oneDPL) supports forward iterators only for a very few algorithms, only for `par` and only in the implementation based on oneTBB.

Returning to the SG1/SG9 feedback, there seemingly are two main reasons why others do not want to restrict parallel algorithms by only random access ranges:

- That would prevent some useful views, such as `filter_view`, from being used with parallel range algorithms.
- That would be inconsistent with the C++17 parallel algorithms.

Given the other aspects of the proposed design, we believe some degree of inconsistency with C++17 parallel algorithms is inevitable and should not become a gating factor for important design decisions.

The question of supporting the standard views that do not provide random access is very important. We think though that it should better be addressed through proper abstractions and new concepts defining iteration spaces, including multidimensional ones, suitable for parallel algorithms. We intend to work on developing these (likely in another paper), however it requires time and effort to make it right, and we think trying to squeeze that into C++26 adds significant risks. For now, random access ranges with known bounds (see § 2.7 Requiring ranges to be bounded) is probably the best approximation that exists in the standard. Starting from that and gradually enabling other types of iteration spaces in a source-compatible manner seems to us better than blanket allowance of any `forward_range`.

§ 2.6. Taking `range` as an output

We would like to propose a range as the output for the overloads that take ranges for input. Similarly, we propose a sentinel for output where input is passed as "iterator and sentinel". See § 4 Proposed API for the examples.

The reasons for that are:

- It creates a safer API where all the data sequences have known limits.
- Not for all algorithms the output size is defined by the input size. An example is `copy_if` (and similar algorithms with *filtering* semantics), where the output sequence is allowed to be shorter than the input one. Knowing the expected size of the output may open opportunities for more efficient parallel implementations.
- Passing a range for output makes code a bit simpler in the cases typical for parallel execution.

It is worth noting that to various degrees these reasons are also applicable to serial algorithms.

There are already range algorithms - `fill`, `generate`, and `iota` - that take a range or an "iterator and sentinel" pair for their output. Their specifics is absence of input sequences, so the output sequence needs a boundary. Nevertheless, these are precedents of specifying output as a range, and extending it from algorithms with zero input sequences to those with one or more seems appropriate.

We think that in practice parallel algorithms mainly write the output data into a container or storage with preallocated space, for efficiency reasons. So, typically parallel algorithms receive `std::begin(v)` or `v.begin()` or `v.data()` for output, where `v` is an instance of `std::vector` or `std::array`. Allowing `v` to be passed directly for output in the same way as for input results in a slightly simpler code.

Also, using classes such as `std::back_insert_iterator` or `std::ostream_iterator`, which do not have a range underneath, is already not possible with C++17 parallel algorithms that require at least forward iterators. Migrating such code to use algorithms with execution policies will require modifications in any case.

All in all, we think for parallel algorithms taking ranges and sentinels for output makes more sense than only taking an iterator.

The main concern we have heard about this approach is the mismatch between serial and parallel variations. That is, if serial range algorithms only take iterators for output and parallel range algorithms only take ranges, switching between those will always require code changes. That can be resolved by:

- (A) adding *output-as-range* to serial range algorithms,
- (B) adding *output-as-iterator* to parallel range algorithms

or both.

The option (A) gives some of the described benefits to serial range algorithms as well; one could argue that it would be a useful addition on its own. The option (B) does not seem to have benefits besides the aligned semantics, while it has the downside of not enforcing the requirements we propose in § 2.7 Requiring ranges to be bounded.

With either (A) or (B), the output parameter for range algorithm overloads could be both a range and an iterator. In the formal wording, this could be represented either as two separate overloads with different requirements on that parameter, or with an exposition-only *range-or-iterator* concept that combines the requirements by logical disjunction, as its name suggest. We did not explore which makes more sense; at glance, there seems to be little practical difference for library implementors.

For "iterator and sentinel" overloads we prefer to always require a sentinel for output, despite the mismatch with the corresponding serial overloads.

§ 2.7. Requiring ranges to be bounded

One of the requirements we want to put on the parallel range algorithms is to disallow unbounded input and output. The reasons for that are:

- First, for efficient parallel implementation we need to know the iteration space bounds. Otherwise, it's hard to apply the "divide and conquer" strategy for creating work for multiple execution threads.
- Second, while serial range algorithms allow passing an "infinite" range like `std::ranges::views::iota(0)`, it may result in an endless loop. It's hard to imagine usefulness of that in the case of parallel execution. Requiring data sequences to be bounded potentially prevents errors at run-time.

We have evaluated a few options to specify such a requirement, and for now decided to use the `sized_sentinel_for` concept. It is sufficient for the purpose and at the same does not require anything that a random access range would not already provide. For comparison, the `sized_range` concept adds a requirement of `std::ranges::size(r)` to be well-formed for a range `r`.

In the case of two or more input ranges or sequences, it is sufficient for just one to be bounded. The other input ranges are then assumed to have at least as many elements as the bounded one. This enables unbounded ranges such as `views::repeat` in certain useful patterns, for example:

```
void normalize_parallel(range auto&& v) {
  auto mx = reduce(execution::par, v, ranges::max{});
  transform(execution::par, v, views::repeat(mx), v, divides);
}
```

At the same time, for an output range (that we propose in § 2.6 Taking range as an output) our preference is to have a boundary independently on the input range(s). The main motivation is to follow established practices of secure coding, which recommend or even require to always specify the size of the output in order to prevent out-of-range data modifications. We think this will not impose any practical limitation on which ranges can be used for the output of a parallel algorithm, as we could not find or invent an example of a random-access writable range which would also be unbounded.

If several provided ranges or sequences are bounded, an algorithm should stop as soon as the end is reached for the shortest one. There are already precedents in the standard that an algorithm takes two sequences with potentially different input sizes and chooses the smaller size as the number of iterations it is going to make, such as `std::ranges::transform` and

`std::ranges::mismatch`. For the record, `std::transform` (including the overload with execution policy) doesn't support different input sizes, while `std::mismatch` does.

§ 2.8. Requirements for callable parameters

In [P3179R0] we proposed that parallel range algorithms should require function objects for predicates, comparators, etc. to have `const`-qualified `operator()`, with the intent to provide compile-time diagnostics for mutable function objects which might be unsafe for parallel execution. We have got contradictory feedback from SG1 and SG9 on that topic: SG1 preferred to keep the behavior consistent with C++17 parallel algorithms, while SG9 supported our design intent.

We did extra investigation and decided that requiring `const`-qualified operator at compile-time is not strictly necessary because:

- The vast majority of the serial range algorithms requires function objects to be `regular_invocable` (or its derivatives), which already has the semantical requirement of not modifying either the function object or its arguments. While not enforced at compile-time, it seems good enough for our purpose because it demands having the same function object state between invocations (independently of `const` qualifier), and it is consistent with serial range algorithms.
- Remaining algorithms should be considered individually. For example, `for_each` using a mutable `operator()` is of less concern if the algorithm does not return the function object (see more detailed analysis below). For `generate`, a non-mutable callable appears to be of very limited use: in order to produce multiple values while not taking any arguments, a generator should typically maintain and update some state.

The following example works fine for serial code. While it compiles for parallel code, users should not assume that the semantics remains intact. Since the parallel version of `for_each` requires function object to be copyable, it is not guaranteed that all `for_each` iterations are processed by the same function object. Practically speaking, users cannot rely on accumulating any state modifications in a parallel `for_each` call.

```
struct callable
{
    void operator()(int& x)
    {
        ++x;
        ++i; // a data race if the callable is executed concurrently
    }
    int get_i() const {
        return i;
    }
private:
    int i = 0;
};

callable c;

// serial for_each call
auto fun = std::for_each(v.begin(), v.end(), c);

// parallel for_each call
// The callable object cannot be read because parallel for_each version purposefully returr
std::for_each(std::execution::par, v.begin(), v.end(), c);
```

```
// for_each serial range version call
auto [_, fun] = std::ranges::for_each(v.begin(), v.end(), c);
```

We allow the same callable to be used in the proposed `std::ranges::for_each`.

```
// callable is used from the previous code snippet
callable c;
// The returned iterator is ignored
std::ranges::for_each(std::execution::par, v.begin(), v.end(), c);
```

Again, even though `c` accumulates state modifications, one cannot rely on that because an algorithm implementation is allowed to make as many copies of `c` as it wants. Of course, this can be overcome by using `std::reference_wrapper` but that might lead to data races.

```
// callable is used from the previous code snippet
// Wrapping a callable object with std::reference_wrapper compiles, but might result in dat
callable c;
std::ranges::for_each(std::execution::par, v.begin(), v.end(), std::ref(c));
```

Our conclusion is that it's user responsibility to provide such a callable that avoids data races, same as for C++17 parallel algorithms.

§ 2.9. Parallel range algorithms are not customization points

We do not propose the parallel range algorithms to be customization points because it's unclear which parameter to customize for. One could argue that customizations may exist for execution policies, but we expect custom execution policies to become unnecessary once the C++ algorithms will work with schedulers/senders/receivers.

§ 2.10. `constexpr` parallel range algorithms

[P2902R0] suggests allowing algorithms with execution policies to be used in constant expressions. We do not consider that as a primary design goal for our work, however we will happily align with that proposal in the future once it progresses towards adoption into the working draft.

§ 3. More examples

§ 3.1. Change existing code to use parallel range algorithms

One of the goals is to require a minimal amount of changes when switching from the existing API to parallel range algorithms. However, that simplicity should not create hidden issues negatively impacting the overall user experience. We believe that the proposal provides a good balance in that regard.

As an example, let's look at using `for_each` to apply a lambda function to all elements of a `std::vector v`.

For the serial range-based `for_each` call:

```
std::ranges::for_each(v, [](auto& x) { ++x; });
```

switching to the parallel version will look like:

```
std::ranges::for_each(std::execution::par, v, [](auto& x) { ++x; });
```

In this simple case, the only change is an execution policy added as the first function argument. It will also hold for the "iterator and sentinel" overload of `std::ranges::for_each`.

The C++17 parallel `for_each` call:

```
std::for_each(std::execution::par, v.begin(), v.end(), [](auto& x) { ++x; });
```

can be changed to one of the following:

```
// Using iterator and sentinel
std::ranges::for_each(std::execution::par, v.begin(), v.end(), [](auto& x) { ++x; });

// Using vector as a range
std::ranges::for_each(std::execution::par, v, [](auto& x) { ++x; });
```

So, here only changing the namespace is necessary, though users might also change `v.begin(), v.end()` to just `v`.

However, for other algorithms more changes might be necessary.

§ 3.2. Less parallel algorithm calls and better expressiveness

Let's consider the following example:

```
reverse(policy, begin(data), end(data));
transform(policy, begin(data), end(data), begin(result), [](auto i){ return i * i; });
auto res = any_of(policy, begin(result), end(result), pred);
```

It has three stages and eventually tries to answer the question if the input sequence contains an element after reversing and transforming it. The interesting considerations are:

- Since the example has three parallel stages, it adds extra overhead for parallel computation per algorithm.
- The first two stages will complete for all elements before the `any_of` stage is started, though it is not required for correctness. If reverse and transformation would be done on the fly, a good implementation of `any_of` might have skipped the remaining work when `pred` returns `true`, thus providing more performance.

Let's make it better:

```
// With fancy iterators
auto res = any_of(policy,
                  make_transform_iterator(make_reverse_iterator(end(data)),
                                          [](auto i){ return i * i; }),
                  make_transform_iterator(make_reverse_iterator(begin(data)),
                                          [](auto i){ return i * i; }),
                  pred);
```

Now there is only one parallel algorithm call, and `any_of` can skip unneeded work. However, this variation also has interesting considerations:

- First, it doesn't compile. We use `transform iterator` to pass the transformation function, but the two `make_transform_iterator` expressions use two different lambdas, and the iterator type for `any_of` cannot be deduced because the types of `transform_iterator` do not match. One of the options to make it compile is to store a lambda in a variable.
- Second, it requires using a non-standard iterator.
- Third, the expressiveness of the code is not good: it is hard to read while easy to make a mistake like the one described in the first bullet.

Let's improve the example further with the proposed API:

```
// With ranges
auto res = any_of(policy, data | views::reverse | views::transform([](auto i){ return i * i
                  pred);
```

The example above lacks the drawbacks described for the previous variations:

- There is only one algorithm call;
- The implementation might skip unnecessary work;
- There is no room for the lambda type mistake;
- The readability is much better compared to the second variation and not worse than in the first one.

§ 4. Proposed API

> NOTE: `std::ranges::for_each` and `std::ranges::transform` are used as reference points. When the design is ratified, it will be spread across other algorithms.

```
// for_each
template <class ExecutionPolicy, random_access_iterator I, sized_sentinel_for<I> S,
          class Proj = identity, indirectly_unary_invocable<projected<I, Proj>> Fun>
  I
    ranges::for_each(ExecutionPolicy&& policy, I first, S last, Fun f, Proj proj = {});
```

```
template <class ExecutionPolicy, random_access_range R, class Proj = identity,
         indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
requires sized_sentinel_for<ranges::sentinel_t<R>, ranges::iterator_t<R>>
  ranges::borrowed_iterator_t<R>
    ranges::for_each(ExecutionPolicy&& policy, R&& r, Fun f, Proj proj = {});

// binary transform with an output range and an output sentinel
template< typename ExecutionPolicy,
          random_access_iterator I1, sentinel_for<I1> S1,
          random_access_iterator I2, sentinel_for<I2> S2,
          random_access_iterator O, sized_sentinel_for<O> SO,
          copy_constructible F,
          class Proj1 = identity, class Proj2 = identity >
requires indirectly_writable<O,
             indirect_result_t<F&, projected<I1, Proj1>, projected<I2, Proj2>>>
         && (sized_sentinel_for<S1, I1> || sized_sentinel_for<S2, I2>)
constexpr binary_transform_result<I1, I2, O>
    transform( ExecutionPolicy&& policy, I1 first1, S1 last1, I2 first2, S2 last2, O result
               F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {} );

template< typename ExecutionPolicy,
          ranges::random_access_range R1,
          ranges::random_access_range R2,
          ranges::random_access_range RR,
          copy_constructible F,
          class Proj1 = identity, class Proj2 = identity >
requires indirectly_writable<ranges::iterator_t<RR>,
             indirect_result_t<F&,
                 projected<ranges::iterator_t<R1>, Proj1>,
                 projected<ranges::iterator_t<R2>, Proj2>>>
         && (sized_sentinel_for<ranges::sentinel_t<R1>, ranges::iterator_t<R1>>
             || sized_sentinel_for<ranges::sentinel_t<R2>, ranges::iterator_t<R2>>)
         && sized_sentinel_for<ranges::sentinel_t<RR>, ranges::iterator_t<RR>>
constexpr binary_transform_result<ranges::borrowed_iterator_t<R1>,
                                  ranges::borrowed_iterator_t<R2>,
                                  ranges::borrowed_iterator_t<RR>>
    transform( ExecutionPolicy&& policy, R1&& r1, R2&& r2, RR&& result, F binary_op,
               Proj1 proj1 = {}, Proj2 proj2 = {} );
```

§ 4.1. Possible implementation of a parallel range algorithm

```
// A possible implementation of std::ranges::for_each
namespace ranges
{
namespace __detail
{
struct __for_each_fn
{
    // ...
```

```
    // Existing serial overloads
    // ...

    // The overload for unsequenced and parallel policies. Requires random_access_iterator
    template<class ExecutionPolicy, random_access_iterator I, sized_sentinel_for<I> S,
             class Proj = identity, indirectly_unary_invocable<projected<I, Proj>> Fun>
                 requires is_execution_policy_v<std::remove_cvref_t<ExecutionPolicy>>
    I
    operator()(ExecutionPolicy&& exec, I first, S last, Fun f, Proj proj = {}) const
    {
        // properly handle the execution policy;
        // for the reference, a serial implementation is provided
        for (; first != last; ++first)
        {
            std::invoke(f, std::invoke(proj, *first));
        }
        return first;
    }

    template<class ExecutionPolicy, random_access_range R, class Proj = identity,
             indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
    ranges::borrowed_iterator_t<R>
    operator()(ExecutionPolicy&& exec, R&& r, Fun f, Proj proj = {}) const
    {
        return (*this)(std::forward<ExecutionPolicy>(exec), std::ranges::begin(r),
                       std::ranges::end(r), f, proj);
    }
}; // struct for_each
} // namespace __detail
inline namespace __for_each_fn_namespace
{
inline constexpr __detail::__for_each_fn for_each;
} // __for_each_fn_namespace
} // namespace ranges
```

§ 5. Absence of some serial range-based algorithms

We understand that some useful algorithms do not yet exist in `std::ranges`, for example, most of generalized numeric operations [numeric.ops]. The goal of this paper is however limited to adding overloads with `ExecutionPolicy` to the existing algorithms in `std::ranges` namespace. Any follow-up paper that adds `<numeric>` algorithms to `std::ranges` should also consider adding dedicated overloads with `ExecutionPolicy`.

§ 6. Further exploration

§ 6.1. Thread-safe views examination

We need to understand better whether using some `views` with parallel algorithms might result in data races. While some investigation was done by other authors in [P3159R0], it's mostly not about the data races but about ability to parallelize pro-

cessing of data represented by various views.

We need to invest more time to understand the implications of sharing a state between `view` and `iterator` on the possibility of data races. One example is `transform_view`, where iterators keep pointers to the function object that is stored in the view itself.

Here are questions we want to answer (potentially not a complete list):

- Do users have enough control to guarantee absence of data races for such views?
- Are races not possible because of implementation strategy chosen by standard libraries?
- Do we need to add extra requirements towards thread safety to the standard views?

§ 7. Revision history

§ 7.1. R1 => R2

- Summarize proposed differences from the serial range algorithms and from the non-range parallel algorithms
- Allow all but one input sequences to be unbounded
- List existing algorithms that take ranges for output
- Update arguments and mitigations for using ranges for output
- Add more arguments in support of random access ranges
- Fix the signatures of `for_each` to match the proposed design

§ 7.2. R0 => R1

- Address the feedback from SG1 and SG9 review
- Add more information about iterator constraints
- Propose `range` as an output for the algorithms
- Require ranges to be bounded

§ 8. Polls

§ 8.1. SG9, Tokyo 2024

Poll 1: `for_each` shouldn't return the callable

SF	F	N	A	SA
2	4	2	0	0

Poll 2: Parallel `std::ranges` algos should return the same type as serial `std::ranges` algos

Unanimous consent.

Poll 3: Parallel ranges algos should require `forward_range`, not `random_access_range`

SF	F	N	A	SA
3	2	3	1	1

Poll 4: Range-based parallel algos should require const operator()

SF	F	N	A	SA
0	7	2	0	0

§ References

§ Informative References

[P2300R9]
Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. *`std::execution`*. 2 April 2024. URL: https://wg21.link/p2300r9

[P2408R5]
David Olsen. *Ranges iterators as inputs to non-Ranges algorithms*. 22 April 2022. URL: https://wg21.link/p2408r5

[P2500R2]
Ruslan Arutyunyan, Alexey Kukanov. *C++ parallel algorithms and P2300*. 15 October 2023. URL: https://wg21.link/p2500r2

[P2902R0]
Oliver Rosten. *constexpr 'Parallel' Algorithms*. 17 June 2023. URL: https://wg21.link/p2902r0

[P3136R0]
Tim Song. *Retiring niebloids*. 15 February 2024. URL: https://wg21.link/p3136r0

[P3159R0]
Bryce Adelstein Lelbach. *C++ Range Adaptors and Parallel Algorithms*. 18 March 2024. URL: https://wg21.link/p3159r0

[P3179R0]
Ruslan Arutyunyan, Alexey Kukanov. *C++ parallel range algorithms*. 15 March 2024. URL: https://wg21.link/p3179r0

[P3300R0]
Bryce Adelstein Lelbach. *C++ Asynchronous Parallel Algorithms*. 15 February 2024. URL: https://wg21.link/p3300r0